

Nutri-MAS: a hybrid multi-agent system for personalized nutritional assistance

Acossi Giorgio Roberto - S5339095
acossigiorgio@gmail.com

July 2026

Project source code: github.com/acossi-giorgio/nutri-mas

1 Details of the proposal

1.1 Proposal specification

Nutri-MAS is a conversational assistant that combines reactive agents, BDI agents, and LLM-based agents. The system collects the user's profile, calculates indicative caloric targets, builds a weekly plan, generates recipes, records the meals actually consumed, and rebalances the subsequent slots when the plan is modified.

1.2 Type of proposal

Type of proposal is creative. The main objective is to experiment with how agents with different decision-making models can collaborate in the same system. *Nutri-MAS* combines three types of agents:

- **BDI agents:** they manage logic based on beliefs and rules, such as planning.
- **LLM agents:** they manage linguistic understanding and textual creativity.
- **Reactive agents:** they manage direct interaction with the user.

1.3 Difficulty level of the proposal

The proposed difficulty level depends greatly on the complexity of the implementation. The complexity of the project does not depend only on the number and quality of the functionalities implemented, but above all on the integration of

heterogeneous agents within a single operational flow. The system must translate requests expressed in natural language into structured commands and orchestrate the collaboration between BDI agents and LLM-based agents, characterized by different decision-making models and reasoning capabilities.

Some simplifications of the domain have been adopted to make the more complex aspects of meal planning manageable. These choices reduce the breadth of the problem, but they do not eliminate the main difficulty, represented by the end-to-end functional integration of the different components of the system. The difficulty of the project has been proposed as *medium-hard*.

2 Introduction

Nutri-MAS is a hybrid multi-agent system for personalized nutritional assistance, meal planning, and recipe generation. This domain requires coordinating different needs: respecting the caloric, dietary, and temporal constraints of the individual user, proposing sufficiently varied recipes, and reacting to deviations from the plan. If, for example, the user consumes a meal different from the one expected, the system must record the event and redistribute the residual budget over the subsequent slots.

Nutri-MAS assigns to BDI agents the logic based on beliefs and rules, such as planning and monitoring; to LLM agents it entrusts linguistic understanding and recipe generation; a reactive agent finally manages access to the system. Through the conversational interface the user can create their own profile, generate and consult a weekly plan, also record off-plan meals, and check the residual caloric budget.

2.1 High-level architecture

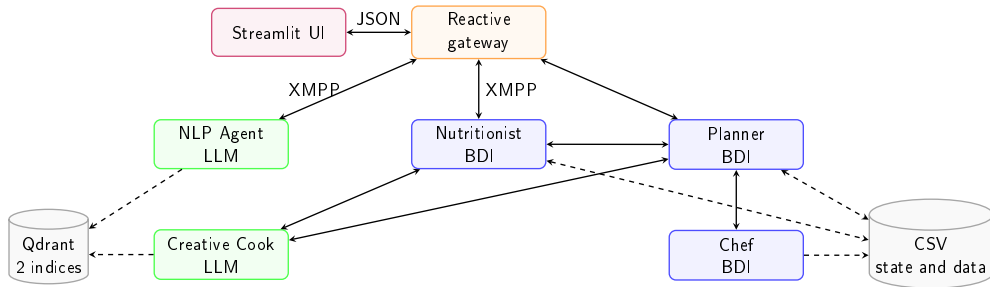


Figure 1: High-level architecture of Nutri-MAS

The architecture of Nutri-MAS is composed of a conversational UI, a Gateway, and a set of specialized agents. The UI communicates with the Gateway by

means of JSON messages. The Gateway connects the UI to the multi-agent system, validates the messages, and controls their routing.

The multi-agent system adopts a hybrid architecture composed of three BDI agents and two LLM-based agents. The BDI agents handle the system’s state and enforce its rules: the Nutritionist manages the user profile, nutritional targets, and dietary logs; the Planner constructs the weekly meal plan; and the Chef selects food templates that satisfy the specified constraints. The LLM-based agents complement these functions: the NLP Agent translates natural-language requests into structured commands, while the Creative Cook turns the templates and constraints provided by the BDI agents into complete recipes.

Communication between the agents takes place through XMPP, while the content of the application messages uses a representation compatible with the AgentSpeak syntax. State persistence is entrusted to CSV files that contain the user profile and the meal plans. Qdrant supports the semantic search of the linguistic examples of the NLP Agent and of the ingredients used by the Creative Cook.

3 Technological stack

The system was developed using Python as the programming language and the SPADE, SPADE-BDI and SPADE-LLM libraries for agent management, Streamlit for the UI, Qdrant for semantic search and LiteLLM as a proxy for access to Large Language Models.

3.1 Streamlit

The UI was implemented with the Python library Streamlit. Communication between the UI and the Gateway takes place through two queues, one carries the user’s messages toward the system, while the other transfers to the UI the responses and events produced by the agents.

The interface keeps message input available even during agent processing. The questions and answers generated by the system preserve an explicit context, used by the NLP Agent to associate the user’s response with the correct AgentSpeak command.

3.2 SPADE

SPADE (Smart Python Agent Development Environment) is a Python framework for the development of distributed multi-agent systems. The framework provides the basic functionalities to define reactive agents, manage their life cycle and allow message exchange through the XMPP protocol. The behaviour of each

agent is defined and organized through behaviours. To these functionalities are added the SPADE-BDI and SPADE-LLM extensions.

SPADE-BDI introduces a deliberative model based on beliefs, desires and intentions, in which the behaviour of agents is defined through AgentSpeak rules and plans.

SPADE-LLM makes it possible to integrate Large Language Models within agents, defining their behaviour through instructions and context and allowing them to use tools to access application functionalities and interact with the other components of the system.

3.3 XMPP

XMPP (Extensible Messaging and Presence Protocol) is an IETF standard for the near-real-time exchange of structured data through XML streams. The protocol defines mechanisms for addressing, authentication and message delivery, identifying each entity through an ID. In Nutri-MAS, each agent possesses a local ID used as identity and communication address. SPADE manages the connection to the XMPP server and the exchange of messages among agents, while the application content of the messages is represented through commands in AgentSpeak syntax.

3.4 SPADE-BDI

The deliberative layer of Nutri-MAS is implemented through SPADE-BDI, an extension that allows the execution of AgentSpeak programs within SPADE agents. The Nutritionist, Planner and Chef agents extend `BDIAgent` and load their respective AgentSpeak programs. In this model, *beliefs* represent the information that the agent considers true, *goals* describe the states that it intends to achieve and plans specify how to react to events. Each plan in fact defines a triggering event, a context that determines its applicability and a set of actions to execute.

Custom Python actions constitute the link between declarative reasoning and application functionalities, such as numerical calculations, access to CSV files, data serialization and the exchange of messages with non-BDI agents. This separation keeps decision-making policies explicit in the AgentSpeak programs, entrusting Python with computational and infrastructural operations.

3.5 SPADE-LLM

SPADE-LLM extends the SPADE agent model by introducing functionalities based on Large Language Models. An LLM agent is configured through a provider, a set of initial instructions (*system prompt*) and a series of tools.

The *tools* are implemented as Python functions and described through a schema that specifies their parameters and methods of use. The model can select the necessary tools, provide the corresponding arguments and use the obtained results to continue processing. A single request can therefore involve multiple interactions between the model and the tools, up to the production of the final result or the reaching of the configured limit.

3.6 LiteLLM Proxy

LiteLLM provides a uniform abstraction layer for access to different LLM providers. In the project it is used as a proxy to expose both the APIs for LLM text generation and those for the production of embeddings. The model used was gpt-5.4 and text-embedding-3-large respectively for text generation and embedding.

3.7 Qdrant

Qdrant is a vector database designed to store embeddings and retrieve contents based on their semantic similarity. Compared to a search based on exact word matching, it makes it possible to identify conceptually close elements even when they are described with different expressions. Nutri-MAS uses it to support the semantic searches of the NLP Agent and the Creative Cook.

4 Data management

Nutri-MAS uses two complementary forms of data management: CSV files, used for persistence and for static datasets, and Qdrant collections, used for semantic search.

4.1 CSV data

To provide the agents with a static knowledge base and preserve the user’s information across different executions of the application, Nutri-MAS uses a persistence system based on CSV files. This solution was preferred to a traditional database in order to reduce infrastructural complexity.

The CSV files are divided into two main categories: persistent files, read and updated during execution, and static datasets, used by the agents in read-only mode. Overall, they perform three functions:

- **Persistence:** preserves the user’s state and history across successive executions.

- **Materialization:** stores the results produced by the planning process.
- **Static knowledge:** provides the agents with food templates, nutritional rules, and ingredient data.

During system startup, each agent loads or indexes the data under its responsibility. The operational and control state of the BDI agents is represented by beliefs. Static reasoning facts, including slot order, slot weights, schedules and slot compatibility, are declared in the AgentSpeak programs; Python actions read those beliefs when they need the same information. The Python code performs adaptation, calculation and persistence functions. Persistent data are written back to the files when the system is shut down.

File	Main agent	Description and use
<code>user.csv</code>	Nutritionist	Contains the user’s profile, including anthropometric data, activity level, goals, allergens, diet type, and nutritional targets.
<code>weight_log.csv</code>	Nutritionist	Stores the weight history.
<code>meal_log.csv</code>	Nutritionist	Records planned and actually consumed meals, together with calories, macronutrients, and registration status.
<code>week_plan.csv</code>	Planner	Contains the complete weekly plan, with recipes, ingredients, instructions, and nutritional values.
<code>week_plan_template.csv</code>	Planner	Stores the selected abstract templates of the active plan, including their target calories, macronutrients, and food categories.
<code>templates.csv</code>	Chef	Contains the candidate food templates, classified by slot, category, and nutritional values.
<code>nutrition_rules.csv</code>	Planner	Defines the variety and frequency constraints associated with food categories, diet types, and slots of the weekly plan.
<code>ingredients.csv</code>	Creative Cook	Contains the ingredients and their nutritional values per 100 grams. The dataset is semantically indexed to generate recipes and estimate freely described meals.

Table 1: Persistent data and reference datasets used by the agents

4.2 Qdrant collections

The vector collections are built at startup from the data and templates available in the project. During execution they are queried by the agents to retrieve the elements semantically closest to their requests.

Collection	Main agent	Description and use
<code>ingredients</code>	Creative Cook	Semantically indexes the names of the ingredients coming from the Australian Food Composition Database [2]. Each ingredient is associated with its calorie, protein, carbohydrate, and fat values.
<code>nlp_translations</code>	NLP Agent	Contains linguistic templates associated with the application messages available in the system. Through semantic search, the NLP Agent identifies the messages most suitable for the user’s request and retrieves their structure from the registry.

Table 2: Vector collections used by Nutri-MAS

The embeddings are generated through the embedding model configured in LiteLLM.

5 Design and implementation of the agents

The following table summarizes the main application functionalities associated with each agent.

Agent	Offered functionalities
Gateway	Integration between the UI and the MAS, including forwarding requests to the NLP Agent for translation, validating commands, and routing them
NLP	Translation of requests in natural language into valid application actions and conversion from AgentSpeak commands to responses in natural language.
Nutritionist	Onboarding, target calculation, weight and meal logging, reminders, recaps and rebalancing of nutritional plans.
Planner	Generation of the nutritional plan, control of variety and frequencies, plan consultation and local replanning of future meals.
Chef	Selection of food templates compatible with diet, slot, calories, macronutrients and constraints.
Creative Cook	Generation of concrete recipes and nutritional estimation of meals described freely.

Table 3: Map between agents and application functionalities

5.1 SPADE behaviours and execution model

5.2 Gateway Agent

The Gateway connects the user interface to the multi-agent system and manages communication in both directions. It sends messages in natural language to the NLP, which translates them into AgentSpeak/Prolog commands; the Gateway then validates and forwards those commands to the recipient agents. In the reverse path, it deterministically converts the technical responses of the agents into validated UI events. It does not make nutritional decisions, but deals only with message flow, validation and presentation.

Its operation is based on two cyclic behaviours. `LocalCommandBehaviour` retrieves from the UI queue the user’s messages and sends them to the NLP for translation; `GatewayBehaviour` receives XMPP messages, forwards translated commands returned by the NLP to the domain agents and publishes domain responses in the UI through the deterministic response renderer.

5.3 NLP Agent

The NLP Agent manages the translation between the natural language of the chat and the AgentSpeak/Prolog commands accepted by the system. User requests are interpreted by an `LLMAgent`; deterministic rendering of BDI responses is instead invoked by the Gateway and does not require a second LLM translation.

The Gateway sends to the NLP Agent one of the following two formats:

Code 1: Input formats of the NLP Agent

```
username: <name>
message: <user text>

username: <name>
question: <question shown to the user>
answer: <user reply>
```

The first format represents an autonomous request, for example “Show me today’s meal plan”. The second is used when the user replies to a previous question. The Gateway includes explicitly both the displayed question and the new answer, making it possible to interpret the user’s replies in the best way, which otherwise may be ambiguous.

A *message* represents an operation that the user can request through the chat. Its definition contains an identifier, a description, the recipient agent, the SPADE performative and the schema needed to build the AgentSpeak performative. All messages are saved in a JSON list, which constitutes the message registry.

Qdrant instead contains short linguistic templates used exclusively for retrieval.

Each chunk associates a possible user formulation with the identifier of a message:

Code 2: Example of indexed linguistic chunk

```
{
  "nl_template": "I ate {actual_food} for {meal_type}.",
  "message": "nutritionist.log_meal"
}
```

Several chunks can point to the same identifier, making it possible to represent different formulations without duplicating the command definition. The search retrieves the semantically closest points, removes duplicate identifiers and returns a number of candidates chosen by the model. The content of the template therefore influences only the initial selection.

For each retrieved identifier, the message registry provides the complete definition.

Code 3: Example of message definition in the registry

```
{
  "id": "nutritionist.get_daily_recap",
  "agent": "nutritionist",
  "message_performative": "achieve",
  "description": "Show today's calorie recap.",
  "signature": "get_daily_recap(Username)",
  "schema": {
    "functor": "get_daily_recap",
    "arity": 1,
    "type": "object",
    "properties": {
      "Username": {
        "type": "string"
      }
    },
    "required": ["Username"]
  }
}
```

The description helps the model distinguish semantically close operations, while signature and schema establish structure, order, types and allowed values. The model therefore must not reconstruct the command from the linguistic chunk, but only from the data of the registry.

The tools available for translation are:

- **search_message_templates**: searches in Qdrant for templates semantically similar to the request and returns the distinct identifiers of the candidate messages. The order of the results indicates only the degree of similarity.
- **get_message**: retrieves from the registry the definitions of the candidate messages, including description, recipient, performative, signature and

argument schema.

- **submit_translation**: validates the ASL syntax, the arity, the absence of free variables and the consistency between command and registered route, then transmits the result to the Gateway.

The translation process follows the following steps:

- **Search**: the model performs semantic search to identify the candidate messages.
- **Selection**: it retrieves the corresponding definitions from the registry and selects the one consistent with the request.
- **Translation**: it fills the arguments with the data provided by the user and invokes **submit_translation**, which validates and transmits the result to the Gateway.

The system prompt describes the translation workflow and the ways of using the tools, without directly including the list of supported messages. The latter are defined in the registry and associated with the linguistic templates indexed in Qdrant. This separation makes the system extensible; in fact, it is possible to add new messages without modifying the system prompt, keeping the general logic of the NLP Agent unchanged.

5.4 Nutritionist Agent

The Nutritionist manages the nutritional state of the user and coordinates the daily cycle of the system. It maintains profile, objectives and logs, starts planning and reacts to events that require confirmations or modifications of the plan. The decision rules are expressed by means of BDI plans.

The beliefs of the Nutritionist represent the user's profile and nutritional objectives, together with the history of meals, weight and the state of the operations in progress. The agent uses them to adapt targets, monitor consumption and decide when to update or rebalance the plan.

The main BDI plans implement the following functionalities:

- **Onboarding and profile management**: **check_user_profile** restores an existing profile or starts the collection of personal data, allergens, diet and culinary preferences. Profile-update plans persist changes and, when necessary, regenerate the weekly plan. In particular, a weight variation of at least 1% causes calories and macronutrients to be recalculated, while reaching the target changes the objective to maintenance. The nutritional calculations are performed by Python actions invoked by these plans.
- **Daily monitoring**: reminder plans react to the **clock_tick** produced by a **CyclicBehaviour** and request confirmation when a meal-slot time is crossed.

Meal-tracking plans then confirm the planned recipe or ask the Creative Cook to estimate a different meal, updating the corresponding log beliefs.

- **Rebalancing:** after a deviation from the plan, `request_rebalance_after` calculates the residual nutritional budget and coordinates the Planner, Chef and Creative Cook to replace only subsequent meals. The beliefs `pending_rebalances` and `pending_rebalance_origin` preserve the workflow state until all responses have arrived.
- **Consultation:** `get_daily_recap`, `get_meal_logs` and `get_weight_logs` read the current beliefs and use serialization actions to build the structured data returned to the Gateway.

5.5 Planner Agent

The Planner transforms the user's nutritional requirement into a weekly plan. It coordinates the construction of the plan, keeps the progress state and waits for the responses of Chef and Creative Cook before proceeding. The planning and control rules are expressed by means of BDI plans.

The beliefs of the Planner represent the received targets, the plan already built, the constraints of variety and the progress state of the planning. The agent consults and updates them to coordinate the choice of meals and avoid repetitions.

The main BDI plans implement the following functionalities:

- **Weekly plan construction:** `build_week_plan` stores the user context and prevents concurrent generations. The Planner derives nutritional targets for the five slot types, obtains compatible template domains from the Chef and builds an assignment over the 35 weekly positions, enforcing daily variety and weekly category constraints. Python actions support the nutritional calculations and feasibility checks, while the BDI plans control the sequence of choices.
- **Recipe generation and plan activation:** once the template assignment is complete, the Planner asks the Creative Cook to produce each concrete recipe and stores the responses in `draft_recipe_row`. Only a complete draft is promoted to `planned_recipe_row`; otherwise the draft is discarded and the previous active plan remains available.
- **Local replanning:** `rebalance_slot` applies the target received from the Nutritionist and coordinates Chef and Creative Cook to rebuild a single meal.
- **Consultation:** `get_plan` reads `planned_recipe_row` and serializes the complete weekly plan for the Gateway.

5.6 Chef Agent

The Chef selects food templates compatible with the constraints of the plan. It does not directly generate the recipes, but narrows the space of alternatives considering diet, slot, category, calories, macronutrients and nutritional constraints. The selection rules are expressed by means of BDI plans and rules.

The beliefs of the Chef represent the available food templates and the criteria with which to evaluate their compatibility with respect to the current request. The agent uses them to filter the alternatives according to diet, slot, category, calories, macronutrients and exclusions. For weekly generation it returns the complete valid domain; for local replanning it returns one strict candidate or reports failure.

The main BDI plans implement the following functionalities:

- **Template selection:** `template_candidate` filters the templates stored in `template_for_plan` according to diet, meal slot, calories, macronutrients and exclusions. Guided requests can also impose a required category. Python actions check numerical tolerances and introduce variability only among candidates already accepted by the BDI rules.
- **Candidate-domain provision:** for each slot-type target received during weekly generation, the Chef returns every valid template and then signals completion of the domain transfer.

5.7 Creative Cook Agent

The Creative Cook deals with managing two distinct operations. During planning it transforms an abstract template into a concrete recipe with name, ingredients, grammages, instructions and macronutrients. During tracking, instead, it interprets a meal described freely and estimates its composition and nutritional values, also producing the template to use for the subsequent rebalancing.

The Creative Cook does not maintain a BDI belief base: it accesses the user's nutritional context, the constraints and targets of the request and the nutritional data of the ingredients. It uses this information to generate compatible recipes or estimate the composition of a free meal.

The Creative Cook receives one of the following two AgentSpeak messages:

Code 4: Input formats of the Creative Cook Agent

```
prepare_plan_slot(Username, Day, MealType, Template, TargetCalories,  
                  TargetProtein, TargetCarbs, TargetFat)  
evaluate_free_meal(Username, Date, MealType, MealName, UserCalories)
```

The first format is sent by the Planner and contains the selected template together

with the targets of the slot. The second comes from the Nutritionist when the user declares having consumed a meal different from the planned one; `UserCalories` can contain a value declared by the user. The signature of the message determines the workflow and the allowed terminal tool, preventing recipe generation and consumption evaluation from being combined.

The available tools are:

- `query_user_nutrition_context`: requests from the Nutritionist, in read-only mode, diet, allergens, culinary preferences, objective, daily calories and meal distribution, the response is handled through a dedicated behaviour.
- `query_ingredient_db`: performs a semantic search in the Qdrant database of ingredients with nutritional values per 100 g.
- `submit_prepared_recipe`: builds a `recipe` message with ingredients and grammages, instructions, calories and macros, then sends it to the Planner.
- `submit_evaluated_free_meal`: builds a `free_meal_evaluated` message with estimated calories, macros and template, then sends it to the Nutritionist.

The process of preparing a recipe follows the following steps:

- **Interpretation**: the model interprets the template and retrieves the nutritional context, checking allergens and diet first and then culinary preferences.
- **Search**: it searches semantically for the necessary ingredients, covering the main groups of the recipe, and uses only ingredients returned by the database.
- **Calculation**: it assigns grammages, estimates calories and macros from the values per 100 g and performs the estimation of portions with respect to the targets.
- **Sending**: it invokes `submit_prepared_recipe` to send the recipe to the Planner.

The process of evaluating a free meal follows the following steps:

- **Search**: the model searches in the database for the full name of the meal, verifying whether there is a correspondence with an already prepared food.
- **Decomposition**: if the result is not sufficient, it decomposes the dish into its main ingredients and searches for them separately.
- **Estimation**: it calculates portions, calories and macronutrients; if the user indicated the calories, it adapts the macros to the declared total.
- **Classification**: it chooses the template that best represents the consumed meal.

- **Sending:** it invokes `submit_evaluated_free_meal` to send the evaluation to the Nutritionist.

The system prompt encodes both the routing and the hierarchy of constraints. Allergens are a strong constraint, the diet must always be respected and culinary preferences are applied only when compatible with safety, template, available ingredients and targets. If no specific preferences are present, the model favors traditional Italian cuisine. The text of the preferences is treated exclusively as recipe data and cannot modify tools, workflow or output format.

Allergens are free text and are treated by the prompt as a strong constraint: in case of doubt the model must avoid the ingredient and choose a safer alternative. The instructions of the recipe remain brief and do not repeat doses or the list of ingredients, already transmitted separately. These constraints entrusted to the model are useful precautions, but they are not equivalent to a deterministic clinical validation. The limit of eight interactions also reduces the risk of search cycles.

5.8 Communication between BDI agents and non-BDI agents

All agents communicate by means of messages transported over XMPP. The difference therefore does not concern the communication protocol, but the way in which each type of agent interprets the received messages. The LLM agents process requests through the SPADE-LLM behaviours, which coordinate the local context of the LLM with the tool calls.

When a normal SPADE agent, such as the Gateway or the Creative Cook, sends a message to a BDI agent, **TellBridge** and **AchieveBridge** adapt the application protocol to AgentSpeak events. Both analyze the ASL body of the message: **TellBridge** translates a **tell** performative into the addition of a belief, while **AchieveBridge** translates an **achieve** performative into an achievement goal.

Between two BDI agents, on the other hand, no bridge is necessary. The AgentSpeak action `.send` provided by SPADE-BDI builds an XMPP message in the native format of the extension and the recipient's **BDIBehaviour** converts it directly into the corresponding event. The bridges therefore remain confined to the connection between BDI and non-BDI agents.

6 Workflow

6.1 Onboarding and first planning

At startup the Nutritionist checks whether a complete user profile exists. If it does, the stored profile is restored and the user is welcomed back without automatically regenerating the plan. If it does not, the Nutritionist collects all required profile data and preferences in a fixed sequence, calculates the goal and daily requirement, and starts generation of the first weekly plan.

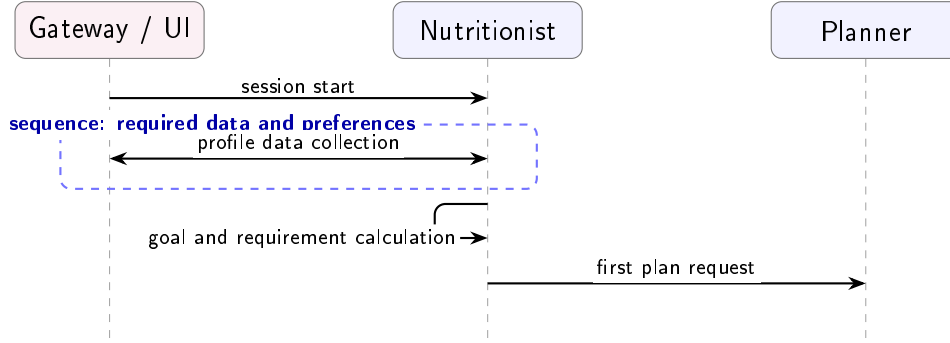


Figure 2: UML diagram of the onboarding workflow

6.2 Construction of the weekly plan

The Nutritionist starts weekly generation after onboarding or an explicit regeneration request. The Planner then coordinates construction of the weekly plan: it computes the nutritional targets and requests from the Chef the complete domain of valid templates for each of the five meal-slot types. The Chef filters its template beliefs according to diet, slot, category, calorie and macronutrient constraints, returning only strict candidates without introducing relaxed fallback templates when no valid candidate is available.

The generation process is divided into two phases. During the first phase, the candidate domains are stored as beliefs and a global AgentSpeak search is performed over the 35 ordered meal slots of the week. Templates are assigned while enforcing daily variety and weekly category-frequency constraints. Whenever a partial assignment becomes infeasible, the relevant choices are reverted and an alternative candidate is explored. This process produces a structured weekly template assignment.

During the second phase, each selected template is passed to the Creative Cook, which turns it into a complete recipe by defining its ingredients, quantities, and preparation instructions. The Planner temporarily stores the generated recipes as beliefs and activates the draft only after all meal slots have been materialized successfully. It then performs a staged replacement: the previous active plan is

cleared only when the complete draft is ready, after which the new recipes and slot targets are promoted and transferred to the Nutritionist as beliefs.

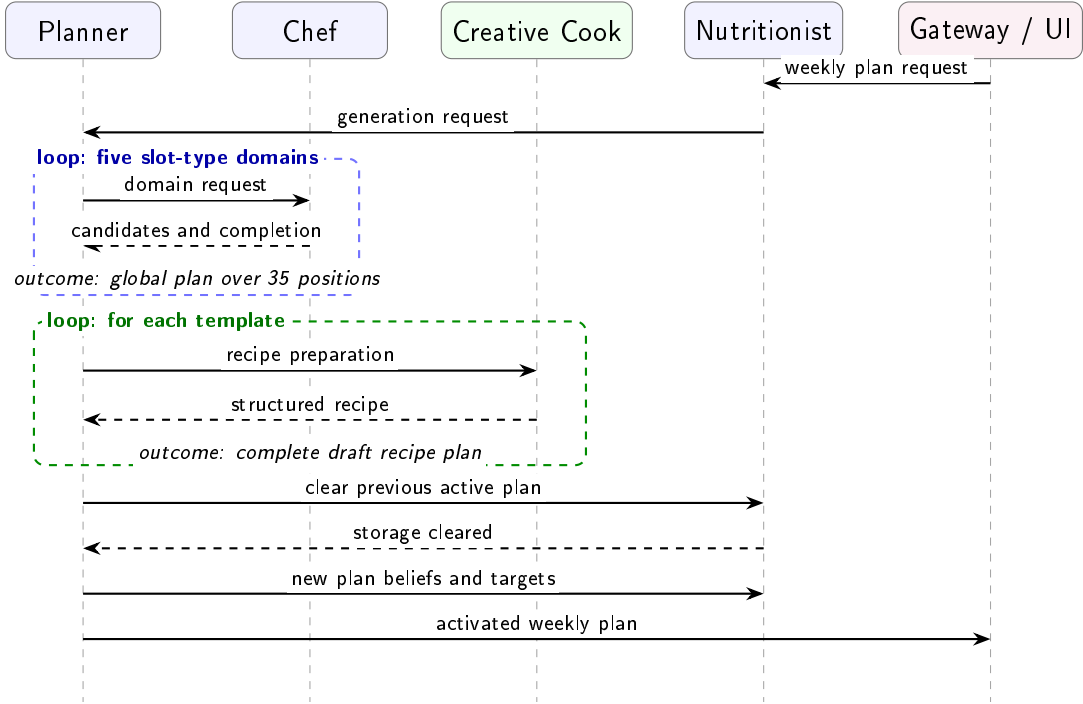


Figure 3: UML diagram of the construction of the weekly plan

6.3 Weight update and plan regeneration

When the user communicates a new weight, the Nutritionist updates the profile and the history and compares the measurement with the weight previously stored in the profile. If the absolute variation reaches 1%, the agent recalculates the requirement on the new weight while maintaining the current goal and regenerates the entire weekly plan; below the threshold it records the value without modifying the plan. For a weight-loss goal the target is reached with a weight less than or equal to the reference; for a gain goal with a weight greater than or equal to it. In that case the goal switches to **maintain**, the requirement is recalculated on the reached weight without deficit or surplus and the week is regenerated.

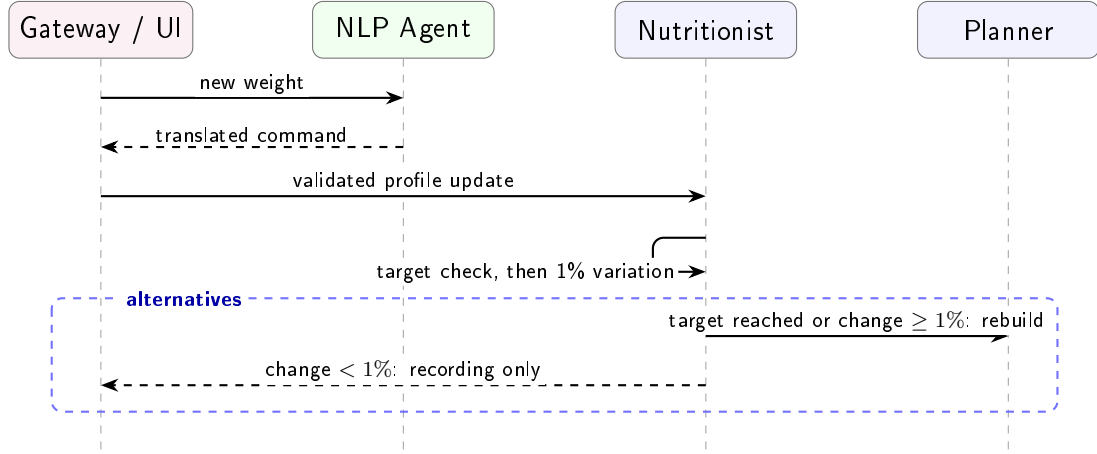


Figure 4: UML diagram of the weight update

6.4 Proactive monitoring, confirmation and different meal

A `CyclicBehaviour` internal to the Nutritionist periodically injects a clock tick into the BDI belief base. The `AgentSpeak` logic compares the current time with that of the previous tick and requests confirmation only for a slot whose time is crossed while the system is running. The check is repeated at every tick for the entire duration of the session. The user can confirm what was planned or indicate that they consumed a different meal. In the latter case, the Cook estimates its nutritional values and the Nutritionist possibly rebalances the subsequent meals of the same day to compensate for any deviation from the plan.

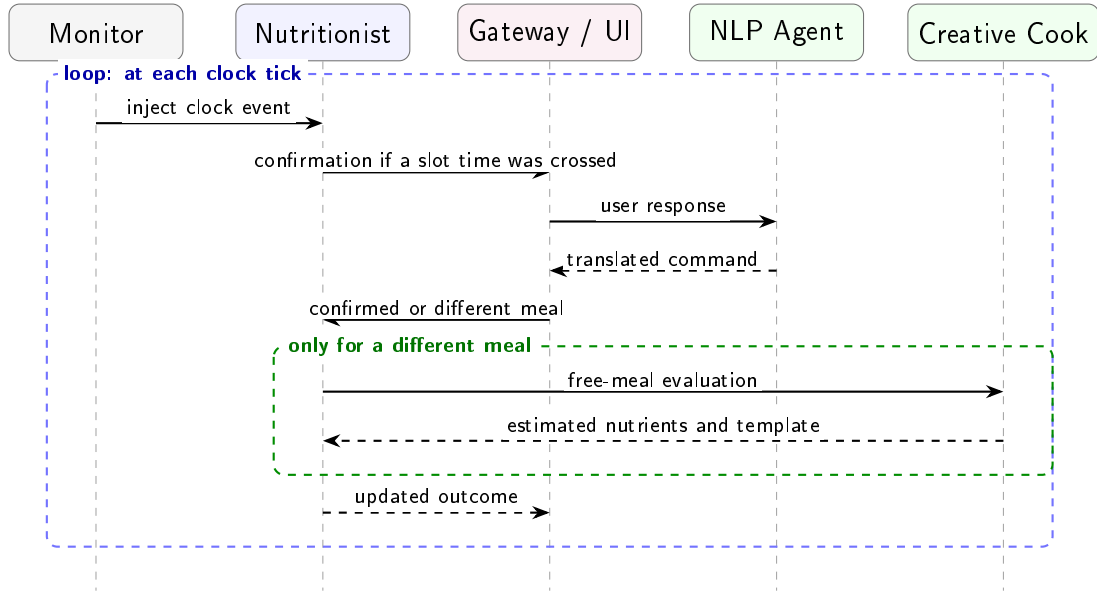


Figure 5: UML diagram of meal monitoring

6.5 Replacement and rebalancing after a deviation

When a different meal is recorded, the Creative Cook first estimates its nutritional values and returns them to the Nutritionist. The Nutritionist compares the actual consumption with the daily requirement and recalculates only the future meals of the same day. For each modifiable future slot, the Planner, Chef and Creative Cook produce a strict replacement compatible with the diet, slot, residual calorie target and scaled macronutrient target. This local procedure does not rerun the global weekly variety and category-frequency search. Compensation is limited by the availability of templates and realistic recipes, the system does not force excessively reduced meals to recover the deviation.

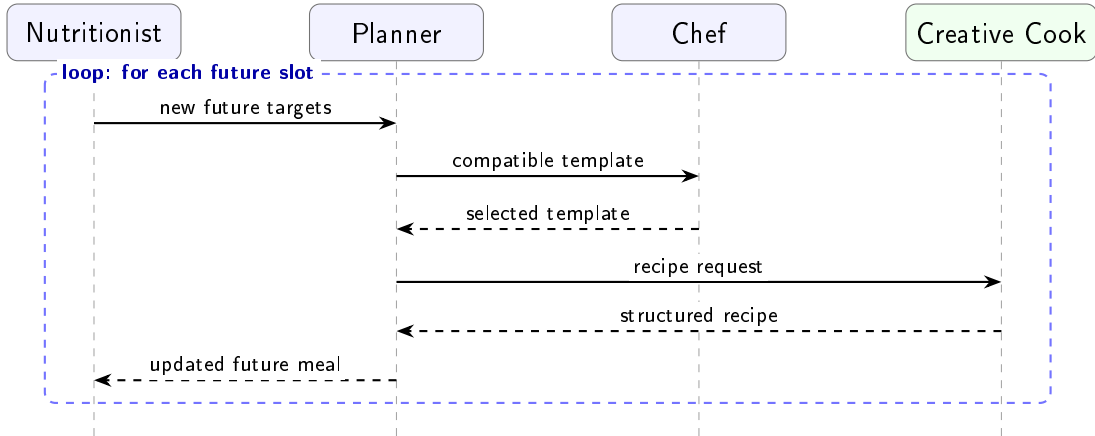


Figure 6: UML diagram of rebalancing

6.6 Plan consultation, recap and history

The user can request the current plan, daily recap and meal or weight history. The NLP translates the natural request and returns the command to the Gateway, which validates and forwards it to the Planner or Nutritionist. The requested information is returned to the Gateway and deterministically rendered in structured format without a second LLM translation.

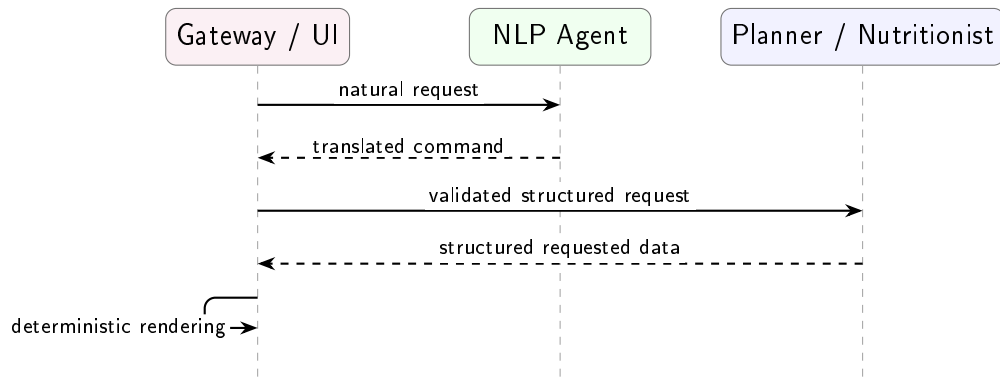


Figure 7: UML diagram of data consultation

7 Demo

Nutri-MAS
What is your height in centimetres?
18:45

You
173
18:45

Nutri-MAS
What is your current weight in kilograms?
18:45

You
93 kg
18:47

Nutri-MAS
How old are you?
18:47

You
I'm 27
18:47

Nutri-MAS
What is your gender? Choose male or female.
18:47

You
Male
18:47

Nutri-MAS
What is your activity level? Choose sedentary, light, moderate, active, or very active.
18:47

You
moderate
18:48

Nutri-MAS
Do you have any allergies?
18:48

You
None
18:48

Nutri-MAS
Which diet do you follow? Choose omnivore, vegetarian, or vegan.
18:48

You
Omnivore
18:48

Nutri-MAS
Do you have culinary preferences or special recipe requests?
18:48

You
I prefer traditional Italian dishes
18:48

Nutri-MAS
Profile completed. Your daily target is 2359 kcal. Your goal is lose. The reference weight is 74.5 kg.
18:48

Nutri-MAS
Great, I am now preparing your weekly meal plan and recipes.
18:48

Figure 8: Onboarding and initial target calculation

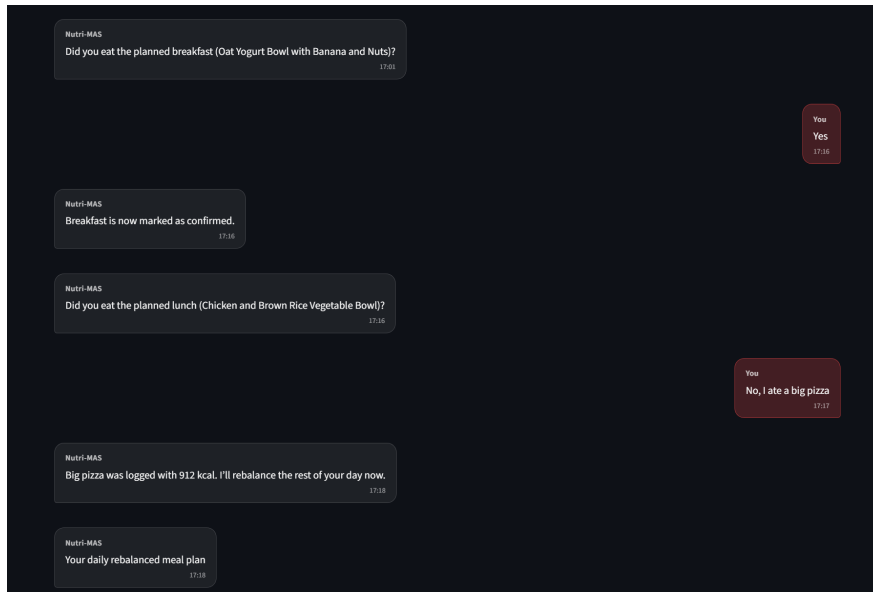


Figure 9: Monitoring and daily rebalancing

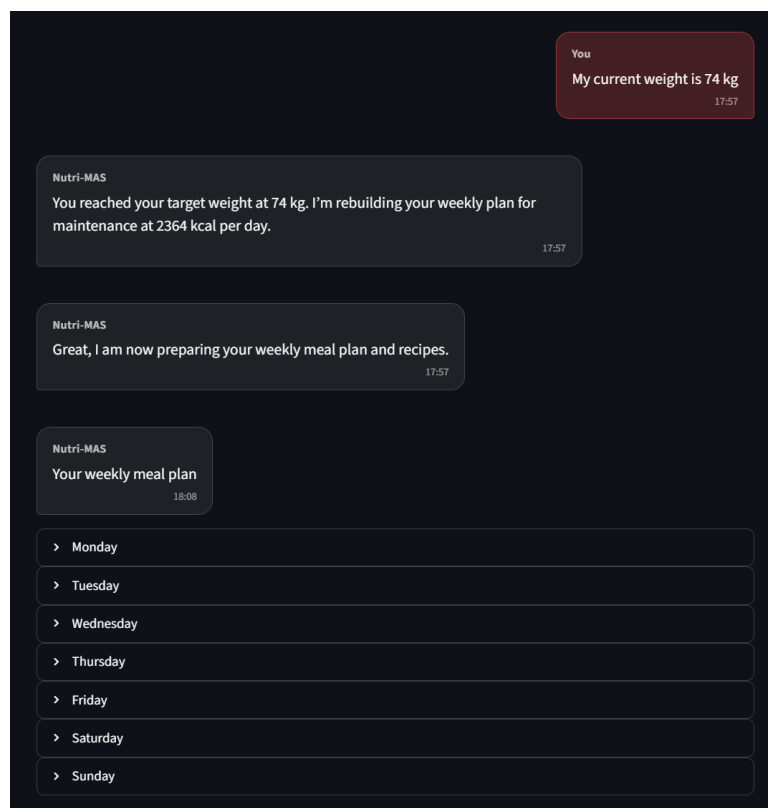


Figure 10: Transition to maintenance after reaching the target

Nutri-MAS
Did you eat the planned lunch (Warm Lentils with Ricotta and Salad)?
13:07

You
No, I ate a big pizza.
13:37

Nutri-MAS
Big pizza was logged with 1003 kcal. I'll rebalance the rest of your day now.
13:55

Nutri-MAS
Your daily rebalanced meal plan
14:50

Thursday

Breakfast

Whole-Wheat Biscuits with Ricotta, Yogurt and Banana
455 kcal · Protein 26 g · Carbs 53 g · Fat 15 g

Ingredients

Ricotta: 120 g, High-protein yogurt: 150 g, Whole-wheat breakfast biscuits: 55 g, Banana: 120 g, Hazelnuts: 10 g

Description and preparation

Crumble the whole-wheat biscuits into a bowl. Top with ricotta, yogurt and sliced banana, then finish with chopped hazelnuts.

Morning snack

Ricotta with Apple and Wholegrain Crispbread
168 kcal · Protein 9 g · Carbs 20 g · Fat 9 g

Ingredients

Ricotta: 70 g, Apple: 100 g, Wholegrain crispbread: 15 g

Description and preparation

Serve the ricotta in a small bowl with sliced apple on the side. Add the crispbread and enjoy as a simple snack.

Lunch

big pizza
1003 kcal · Protein 57 g · Carbs 88 g · Fat 40 g

Afternoon snack

Mini Vanilla Yogurt with Honey
63 kcal · Protein 3 g · Carbs 10 g · Fat 0 g

Ingredients

Apple: 180 g, Ricotta: 100 g, Hazelnuts: 10 g

Description and preparation

Slice the apple and serve it with ricotta. Roughly chop the hazelnuts and sprinkle them on top.

Dinner

Light Vegetable Velouté with Ricotta
170 kcal · Protein 10 g · Carbs 20 g · Fat 6 g

Ingredients

Whole-wheat pasta: 85 g, Ricotta: 170 g, Zucchini: 200 g, Olive oil: 10 g, Parmesan: 8 g

Description and preparation

Cook the pasta until tender. Sauté the zucchini with olive oil, then mix with ricotta and a little cooking water to make a creamy sauce. Toss with the pasta, finish with parmesan, and serve hot.

Write a message...

Figure 11: Rebalancing of the weekly plan after a deviation

You

What i need to eat today

17:27

Nutri-MAS

Your daily meal plan

17:27

▼ Wednesday

Breakfast

Oat Yogurt Bowl with Banana and Nuts

603 kcal · Protein 34 g · Carbs 72 g · Fat 20 g

Ingredients

Oats: 85 g, High-protein yogurt: 300 g, Banana: 180 g, Almonds: 15 g, Walnuts: 10 g, Honey: 15 g

Description and preparation

Stir the oats into the yogurt and let them soften for a few minutes. Slice the banana on top, add the nuts, and finish with honey.

Morning snack

Crispbread with Cottage Cheese and Apple

242 kcal · Protein 15 g · Carbs 28 g · Fat 8 g

Ingredients

Wholemeal crispbread: 35 g, Cottage cheese: 70 g, Apple: 120 g, Pistachios: 8 g

Description and preparation

Spread the cottage cheese over the crispbread. Slice the apple and serve it on the side, then finish with the pistachios.

Lunch

Chicken and Brown Rice Vegetable Bowl

651 kcal · Protein 44 g · Carbs 78 g · Fat 18 g

Ingredients

Chicken breast: 160 g, Brown rice: 90 g, Courgette: 200 g, Mixed leafy greens: 80 g, Carrot: 120 g, Olive oil: 14 g

Description and preparation

Cook the brown rice until tender. Grill the chicken and slice it. Lightly sauté the courgette, serve with the carrot and leafy greens, then top with olive oil and plate everything together.

Afternoon snack

Protein Yogurt with Banana, Oats and Almonds

242 kcal · Protein 19 g · Carbs 29 g · Fat 8 g

Ingredients

High-protein yogurt: 170 g, Banana: 100 g, Oats: 25 g, Almonds: 8 g

Description and preparation

Spoon the yogurt into a bowl. Top with sliced banana, oats and chopped almonds, then serve.

Dinner

Mozzarella with Boiled Potatoes and Salad

654 kcal · Protein 38 g · Carbs 79 g · Fat 22 g

Ingredients

Mozzarella: 150 g, Boiled potatoes: 350 g, Romaine lettuce: 100 g, Carrot: 100 g, Whole-wheat bread: 120 g, Olive oil: 5 g

Description and preparation

Warm the potatoes if needed and slice the mozzarella. Toss the lettuce and carrot with olive oil, then serve with the potatoes and bread.

Write a message...

↑

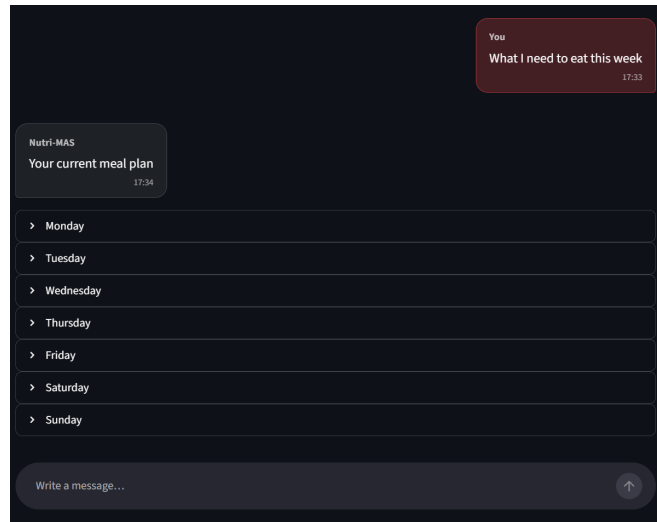


Figure 13: Consultation of the weekly meal plan

8 Limitations and open issues

- **Translation from natural language:** the conversion of the user's requests into structured constraints is based on a finite set of manually defined templates and rules. The system can therefore prove fragile with respect to synonyms, unforeseen formulations, implicit constraints, or ambiguous requests.
- **Limited coverage of meal templates:** the templates used to build meals constitute a selected and manually encoded subset. Consequently, the variety of the generated plans depends on the coverage of the templates and does not include all the possible combinations of ingredients and recipes.
- **Simplification of the nutritional model:** the calculation of calories and macronutrients uses average values coming from the dataset and depends on the matching of ingredients and on the portions adopted. Furthermore, the generation of recipes through LLM can introduce ingredients or quantities not fully consistent with the selected template.
- **Heuristic definition of targets:** the nutritional targets are calculated by means of predefined formulas and coefficients applied uniformly to the different profiles. This approach simplifies the estimation, but does not model all the variables that influence the actual individual requirement.
- **Local caloric rebalancing:** the rebalancing corrects the caloric deviation through a simplified modification of quantities. The procedure does not jointly optimize calories, macronutrients, food frequencies, variety, and compliance with constraints over the entire week.
- **Simplified management of allergens and dietary constraints:** allergens and preferences expressed in free text are translated into instructions for the Cook agent, but a structured representation of the relationships among ingredients, derivatives, food categories, and possible substitutions is not available. This limits the systematic verification of consistency between the declared constraints and the selected ingredients.

9 How to run the project

From the repository root, it is advisable to create and activate a Python virtual environment, then install the dependencies listed in `requirements.txt`:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

Before startup, the two local configuration files must be prepared. The repository

includes example versions without credentials, which can be copied with the following commands:

```
Copy-Item config\.env.example config\.env
Copy-Item config\litellm_config.example.yaml '
    config\litellm_config.yaml
```

To start the entire system, the PowerShell script included in the repository is recommended:

```
.\scripts\run.ps1
```

10 Use of Generative AI

Generative AI tools were used as supporting instruments during the development of the project. Their use was limited to the following activities:

- **Streamlit interface:** AI assistance was used during the implementation of the user interface, allowing more development effort to be dedicated to the multi-agent architecture, BDI reasoning and communication between agents.
- **Debugging support:** AI tools were occasionally used to investigate complex implementation issues. This was particularly useful when debugging AgentSpeak code, where untyped terms, complex predicates, and interactions between concurrent execution flows can make errors difficult to identify.

References

- M. D. Mifflin et al., “A New Predictive Equation for Resting Energy Expenditure in Healthy Individuals”, 1990. Article from which the equation for estimating resting energy expenditure is taken.
<https://doi.org/10.1093/ajcn/51.2.241>
- Food Standards Australia New Zealand, “Australian Food Composition Database – Release 3”, 2025. Dataset from which the nutritional values of the ingredients come. <https://www.foodstandards.gov.au/science-data/food-nutrient-databases/afcd/data-files>
- Centers for Disease Control and Prevention, “Adult BMI Categories”. Reference for the BMI thresholds used in determining the initial goal.
<https://www.cdc.gov/bmi/adult-calculator/bmi-categories.html>